Explications Solution Exo 1 de XSLT :

PS: modele veut dire aussi template ici

Pour tester le XSLT utilisez ce site : https://linangdata.com/xslt-tester

La solution utilise XSLT pour transformer l'arbre XML d'entrée en un nouvel arbre XML qui ne contient que les éléments B et G, avec une racine nommée X. Voici une explication détaillée de chaque partie de cette feuille de style XSLT :

Exo 1: énoncé

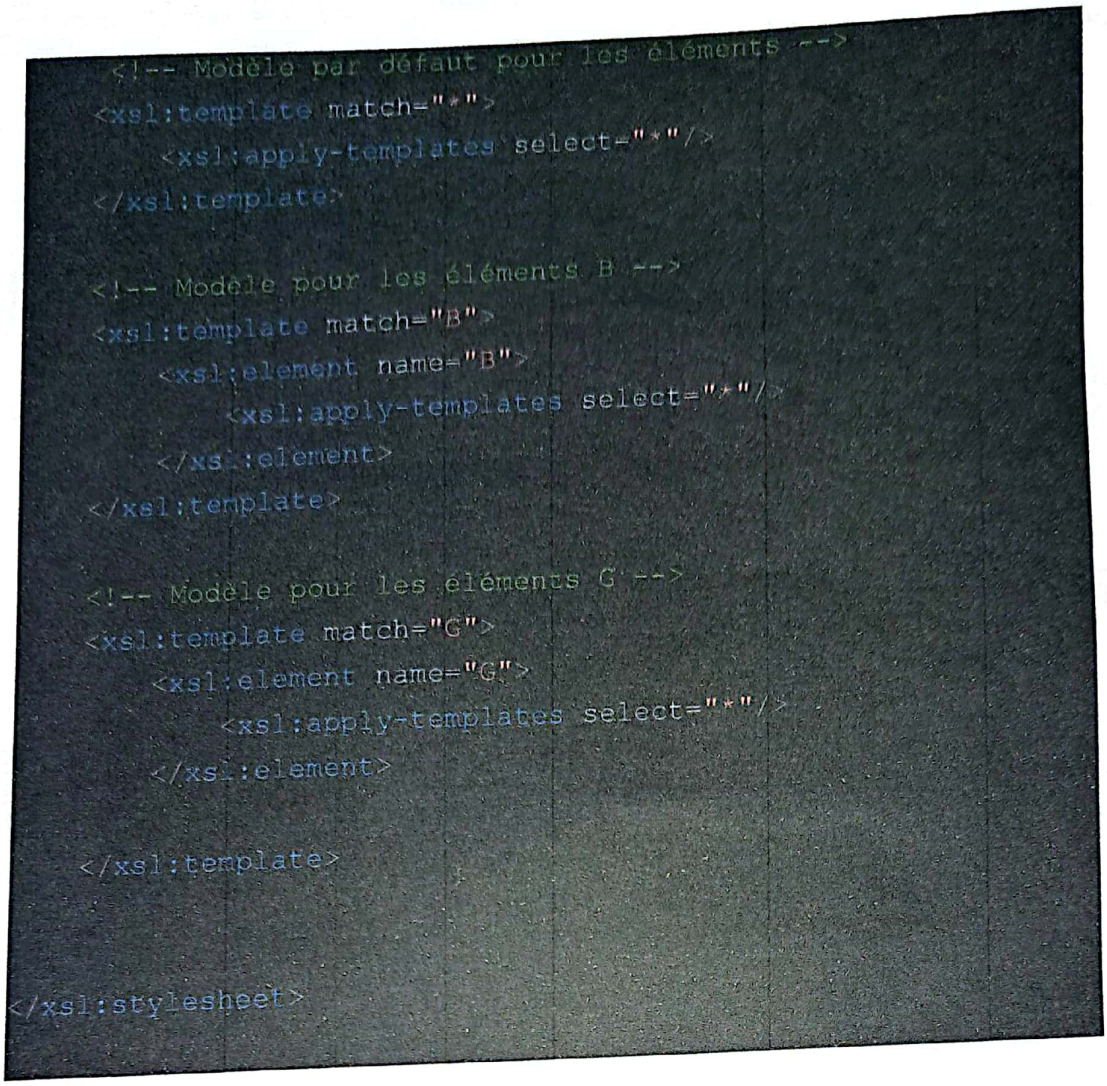
Ecrire une transformation XSLT qui va prendre en entrée un arbre A et produira en sortie un arbre B
constitué uniquement des éléments B et G de l'arbre fourni en entrée. L'arbre B aura pour racine un
nœud nommé X. les lien hiérarchiques (ascendance/descendance) des éléments B et G de l'arbre A
seront préservés dans l'arbre B. L'arbre A est constitué uniquement de nœud type Element.

Feuille de style XSLT Solution:

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
version="1.0">
    <xsl:output method="xml" version="1.0" encoding="UTF-8"
indent="yes"/>

    <!-- Modèle racine -->
    <xsl:template match="/">
        <X>
            <xsl:apply-templates select="*"/>
        </X>
    </xsl:template>
```


CamScanner



```
    <!-- Modèle par défaut pour les éléments -->
    <xsl:template match="*">
        <xsl:apply-templates select="*"/>
    </xsl:template>

    <!-- Modèle pour les éléments B -->
    <xsl:template match="B">
        <xsl:element name="B">
            <xsl:apply-templates select="*"/>
        </xsl:element>
    </xsl:template>

    <!-- Modèle pour les éléments G -->
    <xsl:template match="G">
        <xsl:element name="G">
            <xsl:apply-templates select="*"/>
        </xsl:element>

    </xsl:template>


</xsl:stylesheet>
```

Explication des différentes parties

Directive de sortie
<xsl:output method="xml" version="1.0" encoding="UTF-8" indent="yes"/>

Cette directive spécifie que la sortie doit être un document XML avec l'encodage UTF-8 et doit être indentée pour améliorer la lisibilité. Et le **indent** ne l'oubliez pas, c'est pour qu'il fasse des sauts de lignes et que le résultat soit propre.

Modèle racine

<xsl:template match="/"> <!-- selectionne l'element racine -->
    <X> <!-- créer un élément X -->

CamScanner

```
        <xsl:apply-templates select="*"/>   // continue à lire le code et quand tu trouve un
template pour mon enfant * applique le stp hehe

    </X>
</xsl:template>
```

Ce modèle s'applique à la racine du document XML d'entrée (/).
Il crée un élément racine X dans le document de sortie.
Ensuite, il applique des modèles à tous les enfants de l'élément racine de l'entrée avec **<xsl:apply-templates select="*"/>.**

Modèle par défaut pour les éléments (<xsl:template match="*">):

```
<xsl:template match="*">    // sélectionne les enfants de l'élément actuel
    <xsl:apply-templates select="*"/>    // applique à ses enfants le modèle qui leur
appartient.

</xsl:template>
```

Ce modèle s'applique à tous les éléments de l'entrée (indiqués par *).
Il applique des modèles à tous les enfants de l'élément courant avec <xsl:apply-templates select="*"/>.

// en gros: sélectionne les enfants. Ensuite comme en haut ("sélectionne et applique le bon modèle à ses enfants"). Et puisque chaque enfant va inspecter ses enfants aussi.

Alors pour chaque élément de ce xml on va chercher un modèle (Template) et l'appliquer (si ce template existe pour notre élément).

Modèle pour les éléments B (<xsl:template match="B">):

```
<xsl:template match="B">      // si il trouve un B dans le xml d'entrée
    <xsl:element name="B">     // il crée un élément B dans le document de sortie.
        <xsl:apply-templates select="*"/>   // cherche des modèles à tous les enfants de ce B
    </xsl:element>
</xsl:template>
```

Ce modèle s'applique aux éléments B de l'entrée.
Il crée un élément B dans le document de sortie.
Ensuite, il applique des modèles à tous les enfants de cet élément B avec <xsl:apply-templates select="*"/>.

CamScanner

Modèle pour les éléments G (<xsl:template match="G">):

// la même chose que pour B

Ce modèle s'applique aux éléments G de l'entrée.
Il crée un élément G dans le document de sortie.
Ensuite, il applique des modèles à tous les enfants de cet élément G avec.

Fonctionnement général

En gros, première étape c'est que dans la racine il va créer un X

Ensuite il va parcourir tous ses enfants et leurs enfants... comme on a vu pour leur appliquer un template

Et puisque on a que 2 autres templates pour le B et le G
Alors quand il tombe sur un B ou un G, il va leur appliquer leur template.

Et leur template crée un élément B ou G dans le même endroit de l'enfant
Ainsi, seuls les éléments B et G de l'entrée sont copiés dans la sortie, avec la même structure hiérarchique, et la sortie a une racine X.
Et puisque on a que 2 template donc il va créer que les éléments B et G dans le nouveau XML.

mais ne fait rien pour les éléments qui ne sont pas B ou G. parce qu'on a pas prévu de template pour.

Et donc notre mission est réussie !

Exécution de la transformation

Lorsque cette feuille de style XSLT est appliquée à votre document XML d'entrée, le document XML de sortie aura la structure suivante :

Résultats:



CamScanner



Ainsi, seuls les éléments B et G de l'arbre XML d'entrée sont conservés dans l'arbre XML de sortie, avec leurs relations hiérarchiques intactes et sous une nouvelle racine X.

Les Bases de XSLT

En gros on a un document actuel XML, et XSLT permet de créer un fichier à partir de ce XML.et ce fichier de sortie peut être un autre XML modifié, du HTML, XHTML,SVG... mais on va voir que xml vers xml (On dit aussi qu'il converti)

Et l'astuce ici c'est qu'on applique un **modèle (Template)** sur les éléments du fichier originel pour qu'il ressorte d'une certaine manière dans le fichier de sortie.

Pour commencer il faut créer un fichier par exemple "document.xsl" et se termine par .XSL
Et dans notre fichier y'aura quoi

// on commence par cette ligne car même le xslt c'est une forme de xml
<?xml version="1.0" encoding="UTF-8"?>

// ensuite on a cette ligne qui permet de dire, voila ici ça sera du XSLT et on l'importe et la version. Faudra refermer cette balise à la fin du fichier
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">

// après y'a cette ligne pour spécifier le fichier de sortie en xml
<xsl:output method="xml" version="1.0" encoding="UTF-8" indent="yes"/>

// et ici on commence à mettre nos template et pour créer un modèle on sélectionne sur qui il va être appliquée ? match !
<xsl:template match="nom_element">
// et ici on va mettre ce qui va se passer dans ce modèle
.....
</xsl:template>

CamScanner

```
</xsl:stylesheet>
```

Explication Solution Exo 4 chatgpt

Ce code XSLT effectue une transformation d'un document XML en renversant l'ordre des éléments enfants pour chaque élément du document d'entrée. Voici une explication détaillée de chaque partie du code :

Modèle Racine

xml

Copier le code

```xml
<xsl:template match="/">
    <xsl:apply-templates select="*" />
</xsl:template>
```

- `<xsl:template match="/">` : Ce modèle s'applique à la racine du document XML d'entrée.
- `<xsl:apply-templates select="*"/>` : Applique des modèles à tous les enfants directs de la racine du document.

Modèle pour Tous les Éléments

xml

Copier le code

```xml
<xsl:template match="*">
    <xsl:element name="{name(.)}">
        <xsl:for-each select="*">
            <xsl:sort select="position()" order="descending" />
            <xsl:apply-templates select="." />
        </xsl:for-each>
    </xsl:element>
</xsl:template>
```


CamScanner

- `<xsl:template match="*">` : Ce modèle s'applique à tous les éléments de l'arbre XML.
- `<xsl:element name="{name(.)}">` : Crée un nouvel élément avec le même nom que l'élément courant. `{name(.)}` est une expression qui évalue le nom de l'élément courant.
- `<xsl:for-each select="*">` : Boucle à travers tous les enfants de l'élément courant.
  - `<xsl:sort select="position()" order="descending"/>` : Trie les enfants en ordre décroissant de leur position (renverse l'ordre des enfants).
  - `<xsl:apply-templates select="."/>` : Applique des modèles à chaque enfant sélectionné dans la boucle.

Fermeture des Éléments

xml

Copier le code

```
</xsl:stylesheet>
```

- Cette balise ferme la feuille de style XSLT.

Fonctionnement Global

1. Modèle racine :

2. Modèle pour tous les éléments :

Exemple de Transformation

Document XML d'entrée

CamScanner

```xml
Copier le code
<root>
        <A>
                <B>Content B</B>
                <C>Content C</C>
        </A>
        <D>Content D</D>
</root>
```

Document XML de sortie

```xml
Copier le code
<root>
        <D>Content D</D>
        <A>
                <C>Content C</C>
                <B>Content B</B>
        </A>
</root>
```

Conclusion

Ce code XSLT transforme le document XML en renversant l'ordre des enfants pour chaque élément, tout en préservant les noms des éléments et leurs contenus.

CamScanner